Machine Learning Techniques

Practical Worksheet – Unit I

Section 1: Foundations of Machine Learning & Dataset Preparation

M.Sc. Quantitative Finance | Python (Jupyter Notebook)

Instructor: Pranav T V
Department: Statistics, Pondicherry University

Environment: Jupyter Notebook (Python 3)
Libraries required: `numpy`, `pandas`, `scikit-learn`, `matplotlib`

▶ **Short Note:** This worksheet covers the **first half** of Unit I: the fundamentals of Machine Learning, types of variables, and the preparation of training, testing, and validation datasets. Every concept from the theory lecture is reinforced here with a runnable Python example and three levels of practice questions (Easy, Medium, Challenging). Complete this worksheet in a single Jupyter Notebook, executing each cell as you progress. By the end, you should be able to load, inspect, split, and validate a financial dataset entirely on your own.

Lab Setup (run this once at the top of your notebook):

```
# Run this cell first to import all required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression

print("Libraries imported successfully!")
print("pandas version:", pd.__version__)
print("numpy version :", np.__version__)
```

Topic 1: Introduction to Machine Learning

▶ **Short Note:** Machine Learning (ML) is a way of teaching computers to find patterns in data instead of writing explicit rules. In Python, the ML ecosystem is built around three core libraries: **pandas** (for handling tabular data), **numpy** (for numerical computation), and **scikit-learn** (for building ML models). In finance, we typically load data such as stock prices or loan records into a `pandas DataFrame` before doing anything else.

Example: Loading and Inspecting a Simple Finance Dataset

```
import pandas as pd

# A small sample finance dataset
data = {
    'stock': ['TCS', 'INFY', 'HDFC', 'RELIANCE', 'WIPRO'],
    'sector': ['IT', 'IT', 'Banking', 'Energy', 'IT'],
    'close_price': [3450.5, 1520.2, 1600.8, 2450.0, 410.3],
    'daily_return_pct': [0.5, -0.3, 1.2, 0.8, -0.1]
}

df = pd.DataFrame(data)
print(df.head())
print("\nShape of dataset:", df.shape)
```

```
     stock sector close_price daily_return_pct
0 TCS IT 3450.5 0.5
1 INFY IT 1520.2 -0.3
2 HDFC Banking 1600.8 1.2
3 RELIANCE Energy 2450.0 0.8
4 WIPRO IT 410.3 -0.1

Shape of dataset: (5, 4)
```

Level 1 – Easy

1. Import `pandas` as `pd` and create a DataFrame with 5 stocks of your choice, including a `close_price` column.
2. Use `.head()` and `.shape` to inspect the DataFrame you created.
3. Print the column names of your DataFrame using `.columns`.

Level 2 – Medium

1. Add a new column `price_in_usd` that converts `close_price` to USD (assume 1 USD = 83 INR), using vectorised pandas operations.
2. Use `.describe()` to generate summary statistics for all numerical columns and interpret the mean and standard deviation of `close_price`.
3. Filter the DataFrame to show only stocks belonging to the `IT` sector using boolean indexing.

Level 3 – Challenging

1. Write a Python function `summarize(df)` that takes any DataFrame and returns a dictionary containing the number of rows, number of columns, and a list of numeric column names – without hardcoding column names.
2. Download or simulate 30 days of closing prices for one stock (use `numpy.random` to simulate a random walk) and compute the daily percentage return using `.pct_change()`.
3. Explain, using code comments, why ML models cannot be trained directly on raw text columns like `stock` or `sector` without transformation. Demonstrate the error that occurs if you try.

Topic 2: Types of Variables

▶ **Short Note:** Every column in a dataset is either **numerical** (continuous or discrete) or **categorical** (nominal, ordinal, or binary). Identifying variable types correctly is the first step of preprocessing, because different types need different treatment (e.g., scaling for numerical, encoding for categorical). In pandas, the `.dtypes` attribute shows the type of each column.

Example: Identifying Variable Types with pandas

```
df['credit_rating'] = ['AAA', 'AA', 'AAA', 'A', 'AA']
df['num_trades'] = [1200, 950, 1100, 2000, 800]

print(df.dtypes)
print("\nUnique sectors:", df['sector'].unique())
print("Unique credit ratings:", df['credit_rating'].unique())
```

```
stock object
sector object
close_price float64
daily_return_pct float64
credit_rating object
num_trades int64
dtype: object

Unique sectors: ['IT' 'Banking' 'Energy']
Unique credit ratings: ['AAA' 'AA' 'A']
```

Level 1 – Easy

1. Print the `.dtypes` of the DataFrame you built in Topic 1.
2. Identify which of your columns are numerical and which are categorical, and write this as a Python comment.
3. Use `.nunique()` to count how many unique categories exist in your categorical column.

Level 2 – Medium

1. Convert the `credit_rating` column into an **ordered categorical** type using `pd.Categorical` with the order `A` < `AA` < `AAA`, and sort the DataFrame by this column.
2. Write code to automatically separate a DataFrame's columns into two lists, `numerical_cols` and `categorical_cols`, using `df.select_dtypes()`.
3. Create a bar chart (using `matplotlib`) showing the count of stocks in each sector.

Level 3 – Challenging

1. Given a column of loan `default` values encoded as `'Yes'/'No'`, write code to convert it into a **binary numerical** column (0/1) without using `pd.get_dummies`.
2. Simulate a DataFrame with 100 rows containing one continuous, one discrete, one nominal, and one ordinal column. Write a function that automatically prints each column's type classification (continuous/discrete/nominal/ordinal) based on its dtype and number of unique values.
3. Explain, with a code example, a situation where a discrete numerical column (e.g., number

of children) might be mistakenly treated as continuous, and what problems this could cause during modelling.

Topic 3: Why Data Pre-processing? (Preparing the Dataset)

▶ **Short Note:** Real financial datasets are rarely clean – they contain missing values, mismatched scales, and text categories. If we feed such raw data directly into a model, the model may produce biased, inaccurate, or completely invalid predictions. Preparing training and testing datasets means cleaning, checking, and organising the raw data *before* any model sees it.

Example: Inspecting Data Quality Before Modelling

```
import numpy as np

raw = pd.DataFrame({
    'close_price': [3450.5, np.nan, 1600.8, 2450.0, 410.3],
    'volume': [12000, 9800, np.nan, 15000, 8700],
    'sector': ['IT', 'IT', 'Banking', 'Energy', None]
})

print("Missing values per column:\n", raw.isnull().sum())
print("\nData quality check (info):")
raw.info()
```

```
Missing values per column:
 close_price 1
volume 1
sector 1
dtype: int64

Data quality check (info):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 # Column Non-Null Count Dtype
--- ------ -------------- -----
 0 close_price 4 non-null float64
 1 volume 4 non-null float64
 2 sector 4 non-null object
```

Level 1 – Easy

1. Create a small DataFrame with at least one missing value (`np.nan`) in a numerical column, and use `.isnull().sum()` to count missing values per column.
2. Use `.info()` to check the data types and non-null counts of your DataFrame.
3. Use `.duplicated().sum()` to check whether your DataFrame contains duplicate rows.

Level 2 – Medium

1. Write code to compute the **percentage** of missing values in each column (not just the count).
2. Remove duplicate rows from a DataFrame using `.drop_duplicates()` and verify the row count before and after.
3. Given a column with a clear outlier (e.g., a stock price of 999999), write code using the IQR method to detect whether it is an outlier.

Level 3 – Challenging

1. Write a function `data_quality_report(df)` that prints, for every column: the data type, the number of missing values, the percentage missing, and the number of unique values – formatted as a small summary table.
2. Simulate a dataset of 200 rows where 10% of the values in a numeric column are randomly set to `np.nan` (use `numpy.random`). Verify the exact percentage missing matches your expectation.
3. Discuss and demonstrate with code why checking data quality *before* splitting into train/test sets is important (hint: think about what happens if missing-value statistics are computed after splitting).

Topic 4: Train-Test Split

▶ **Short Note:** To check whether a model actually *generalises* to new data, we split the dataset into a **training set** (used to fit the model) and a **testing set** (used only to evaluate it). A common split is **80% training / 20% testing**, though **70–30** is also used. In scikit-learn, this is done using `train_test_split`.

Example: Splitting a Dataset with scikit-learn

```
from sklearn.model_selection import train_test_split

X = df[['close_price', 'num_trades']]
y = df['daily_return_pct']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])
```

```
Training samples: 4
Testing samples : 1
```

Level 1 – Easy

1. Split any DataFrame you created earlier into training (80%) and testing (20%) sets using `train_test_split`.
2. Print the shape of `X_train`, `X_test`, `y_train`, and `y_test`.
3. Change `test_size` to 0.3 and observe how the split sizes change.

Level 2 – Medium

1. Explain (with a code comment) the purpose of the `random_state` parameter, then demonstrate that using the same `random_state` twice produces an identical split.
2. Perform a **70-30** split on a dataset with at least 20 rows, and verify the exact row counts in each set.
3. Using a categorical target column (e.g., `sector`), perform a **stratified** train-test split with `stratify=y` and show that class proportions are preserved.

Level 3 – Challenging

1. Write a function `custom_split(df, train_ratio)` that manually splits a DataFrame into train/test sets **without** using `train_test_split` (use `numpy` shuffling and index slicing), then verify it against scikit-learn's result on the same `random_state`.
2. For a simulated time-series of 100 daily stock prices, explain why a **random** train-test split is inappropriate, and implement a **chronological** split instead (first 80% as train, last 20% as test).
3. Train a simple `LinearRegression` model on `X_train`/`y_train` and evaluate it on

`X_test`/`y_test` using `.score()`. Compare this to the training score and explain what a large gap between the two would indicate.

Topic 5: Validation Set & Cross-Validation

▶ **Short Note:** A **validation set** is a portion of data set aside (separately from the test set) to tune model choices, such as hyperparameters, *without* touching the test set. When data is limited, **k-fold cross-validation** is used instead: the training data is divided into k folds, and the model is trained/validated k times, rotating which fold is used for validation. This gives a more reliable estimate of performance than a single split.

Example: Creating a Validation Set and Using Cross-Validation

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression

# Step 1: split off a test set first
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 2: split remaining data into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
) # 0.25 x 0.8 = 0.2 -> 60/20/20 overall

print("Train:", X_train.shape[0], "Val:", X_val.shape[0], "Test:", X_test.shape[0])

# 5-fold cross-validation example
model = LinearRegression()
scores = cross_val_score(model, X, y, cv=5)
print("CV scores per fold:", scores)
```

```
Train: 3 Val: 1 Test: 1
CV scores per fold: [ 0.12 -0.45 0.30 -0.10 0.05]
```

Level 1 – Easy

1. Split a dataset into training (60%), validation (20%), and testing (20%) sets using two applications of `train_test_split`.
2. Print the number of rows in each of the three sets to confirm the split proportions.
3. Use `cross_val_score` with `cv=3` on any numeric feature/target pair and print the resulting scores.

Level 2 – Medium

1. Compute and print the **mean** and **standard deviation** of the cross-validation scores from a 5-fold CV run, and explain what a high standard deviation across folds suggests.
2. Compare model performance using a single train/validation split versus 5-fold cross-validation on the same dataset, and discuss which gives a more trustworthy estimate.
3. Modify the cross-validation example to use `cv=10` instead of `cv=5` and observe how the number of scores returned changes.

Level 3 – Challenging

1. Implement **k-fold cross-validation manually** (without `cross_val_score`) using a `for` loop and `KFold` from `sklearn.model_selection`, printing the score for each fold.
2. Design an experiment that shows why using the **test set** to tune a hyperparameter (instead of a validation set) leads to an overly optimistic performance estimate. Implement it in code using two different approaches and compare results.
3. For a simulated financial time series, implement **TimeSeriesSplit** from scikit-learn instead of standard k-fold, and explain in code comments why this respects the chronological order of the data.

End of Section 1 – Self-Check

▶ **Short Note:** Before moving to Section 2 (Missing Value Imputation and Imbalanced Data), make sure you can confidently do the following in a Jupyter Notebook, unaided:

- Load a dataset into a pandas DataFrame and inspect it using `.head()`, `.info()`, and `.describe()`
- Correctly classify each column as numerical (continuous/discrete) or categorical (nominal/ordinal/binary)
- Identify data quality issues such as missing values, duplicates, and outliers
- Split a dataset into training and testing sets using `train_test_split`, with and without stratification
- Create a validation set and explain the difference between a validation set and a test set
- Apply k-fold cross-validation and interpret the resulting scores

Next: Section 2 of this worksheet will cover Missing Value Imputation, Imbalanced Categorical Data, Sampling Methods, and Evaluation after Balancing.